

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics
COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I **JAIVANT (192311326)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *JAIVANT.S*

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of	Good understanding	Partial understanding	Poor understanding

		underlying big data architecture			
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

ANSWERS

1. Recommendation Engine

Introduction

A recommendation engine analyzes user activities such as clicks, searches, and purchases to understand their interests and provide personalized suggestions. Since the system receives data from multiple devices, it needs a scalable big data architecture.

Probable Data Types

The system would handle different types of data:

- **Structured data:** Customer profiles, product details, and purchase records.
- **Semi-structured data:** Clickstream events, search queries, and application logs.
- **Unstructured data:** Product reviews, comments, and feedback.

Data Ingestion Flow

Data would be collected from websites, mobile applications, and other devices. A streaming platform such as **Apache Kafka** could continuously collect click and search events. Historical purchase information could be transferred using batch processing.

Storage and Processing

The collected data could be stored in a **data lake or distributed storage system** such as HDFS or cloud object storage. Apache Spark could then clean, transform, and process the data.

Analytics Stage

Machine learning algorithms can analyze browsing history, searches, and previous purchases to identify user preferences. The system can then generate personalized product recommendations.

Likely Design Decisions

Real-time processing would be useful for capturing the user's latest activity, while batch processing can handle historical data. Distributed storage and processing would allow the system to support millions of users.

Conclusion

Overall, the recommendation engine is likely to combine **streaming ingestion, distributed storage, big data processing, and machine learning** to provide accurate and personalized recommendations.

2. Ride-Sharing Live Traffic and Demand Forecasting

Introduction

A ride-sharing application needs to process location and trip information continuously to display live traffic conditions and predict future ride demand.

Probable Data Sources

The system would collect:

- GPS locations from vehicles.
- Ride requests and completed trips.
- Traffic and road-condition information.
- Historical trip information.
- Weather information, if required.

Distributed Storage

Large amounts of GPS and trip data can be stored using **HDFS or cloud object storage**. Distributed storage allows data to be divided across multiple machines and provides scalability and fault tolerance.

Data Ingestion and Processing

Apache Kafka can collect GPS locations and ride requests continuously. A processing framework such as **Apache Spark Streaming** can analyze this incoming data with low delay.

Traffic Analysis

The system can analyze the current locations and speeds of vehicles to identify traffic congestion. The results can then be displayed as a live traffic heatmap in the application.

Demand Forecasting

Historical ride requests can be analyzed using machine learning to identify demand patterns. The system can predict areas where more passengers are likely to request rides in the near future.

Likely Design Decisions

Real-time processing is necessary for live traffic information, while historical data is important for accurate demand forecasting. Distributed storage allows the platform to handle data from thousands or millions of rides.

Conclusion

The likely architecture combines **distributed storage, streaming ingestion, real-time processing, and machine learning** to provide live traffic information and accurate demand forecasts.

3. Fraud Detection Dashboard

Introduction

A fraud detection system must identify suspicious card transactions as quickly as possible. Therefore, it requires real-time data processing along with strong security and automated alerting.

Probable Data Sources

The system would process:

- Transaction amount and time.
- Merchant information.
- Transaction location.
- Customer and card information.
- Device information.
- Previous transaction history.

Streaming Mechanism

Transactions can be continuously collected using **Apache Kafka** or a similar streaming platform. This allows transactions to reach the analytics system almost immediately after they occur.

Fraud Analytics

A stream-processing system such as **Apache Spark** can analyze transactions in near real time. Rule-based methods can detect obvious suspicious activities, while machine learning can identify unusual transaction patterns.

For example, an unusually large transaction or transactions occurring in different locations within a very short period could receive a high fraud score.

Security Mechanisms

Financial information requires strong protection. The system can use:

- **Encryption** to protect data during storage and transmission.
- **Role-based access control** to restrict access.
- **Strong authentication** for authorized users.
- **Data masking or tokenization** to protect card information.
- **Audit logs** to track access and activities.

Alerting System

When a transaction receives a high risk score, the system can automatically generate an alert on the fraud detection dashboard. Security personnel can then investigate the transaction or take appropriate action.

Likely Design Decisions

Near-real-time processing is selected because fraud needs to be detected quickly. Automated alerts reduce response time, while security controls protect sensitive financial information.

Conclusion

The system most likely combines **real-time streaming, fraud analytics, machine learning, security controls, and automated alerting** to detect suspicious transactions effectively.

4. Social Media Platform

Introduction

A social media platform handles a wide variety of information, including videos, comments, hashtags, and user engagement data. Since these datasets have different formats, multiple storage and processing methods are required.

Types of Data

The platform would manage:

- **Structured data:** User profiles, likes, shares, and engagement statistics.
- **Semi-structured data:** Hashtags, application logs, and event records.
- **Unstructured data:** Videos, images, comments, and other media.

Data Storage Strategy

Structured information can be stored in relational or NoSQL databases. Large videos and images can be stored in **distributed object storage or a data lake**.

This approach allows the platform to store large files without depending on a single storage server.

Data Integration

User interactions such as likes, shares, comments, and views are continuously generated. A streaming system such as **Apache Kafka** can collect these events from different services and applications.

Data Processing and Analytics

Apache Spark or similar big data tools can process engagement data. The platform can use analytics to identify:

- Trending hashtags.
- Popular videos.
- User engagement patterns.
- Frequently shared content.

Text analytics can also be applied to comments to understand user opinions and sentiment.

Likely Design Decisions

Different storage systems are used because structured records and large media files have different requirements. Distributed storage provides scalability, while streaming processing allows the platform to analyze rapidly changing engagement data.

Conclusion

The platform is likely to use a combination of **relational/NoSQL databases, distributed object storage, streaming systems, and big data analytics** to manage its large and diverse datasets.

5. Manufacturing IoT Platform

Introduction

A manufacturing IoT platform collects information from machines located across many plants. Since machines continuously generate sensor readings and logs, the system needs an efficient way to integrate, store, and analyze this information.

Probable Data Sources

The main sources include:

- Machine sensor readings.
- Equipment logs.
- Maintenance records.
- Production information.
- Machine failure events.

Integration Tools

IoT gateways can collect information directly from machines. A streaming platform such as **Apache Kafka** can then transfer real-time sensor events from different plants to the central system.

ETL or data-integration tools can combine sensor data with maintenance and production records.

Storage Strategy

A **distributed data lake** can store large volumes of machine logs and sensor data. Time-series databases can be used for continuously changing sensor readings, while structured maintenance records can be stored in databases or warehouses.

Analytics Path

The collected data can be cleaned and processed using Apache Spark. Analytics can identify abnormal machine behavior, changes in temperature, vibration, pressure, or other important parameters.

Predictive Maintenance

Machine learning models can analyze historical sensor and maintenance data to predict possible equipment failures. Maintenance teams can then service the machine before a major breakdown occurs.

Likely Design Decisions

Streaming processing is useful for real-time machine monitoring, while distributed storage is required because multiple plants continuously generate large amounts of data. Historical data is retained to improve the accuracy of predictive models.

Conclusion

The manufacturing platform is likely to combine **IoT gateways, streaming integration, distributed storage, big data processing, and predictive analytics**. This can improve machine reliability and reduce unexpected production downtime.
